Irish News Dataset Analysis

Complete Report: Questions 1-9

R Programming Solutions with Explanations

Dataset Overview

The dataset contains Irish news articles with the following five columns:

Column	Type	Description
publish_date	chr	Date of publication (e.g., 'Saturday, 14th of November, 2009')
headline_category	chr	Category of the article (e.g., news, opinion, sport, business, lifestyle, culture)
headline_text	chr	The headline text of the article
news_provider	chr	News source (e.g., Irish Times, RTE News, TheJournal.ie)
engagement_score	dbl	Numeric score representing reader engagement (some values are NA)

General Assumptions

Before solving the questions, the following assumptions were made about the dataset:

1. The dataset is loaded into R as a dataframe named 'ds'.
2. The publish_date column contains dates in the format 'Weekday, Nth of Month, Year' (e.g., 'Saturday, 14th of November, 2009'). Ordinal suffixes (st, nd, rd, th) need to be removed before parsing.
3. The engagement_score column contains some NA (missing) values. These must be handled appropriately in each analysis.
4. The headline_category column has a hierarchical naming structure using dots (e.g., 'news.law.courts.circuit-court'). There are 80+ unique subcategories.
5. Libraries dplyr, lubridate, ggplot2, and tidyr are available and installed.

Question 1: Unique Values in headline_category

Problem Statement

How many unique values are there in the headline_category column of the data file?

Approach

We use the unique() function to extract all distinct values from the headline_category column, then use length() to count them.

R Code

```
# Count the number of unique values in headline_category
length(unique(ds$headline_category))

# Display all unique category names sorted alphabetically
sort(unique(ds$headline_category))
```

Explanation

The unique() function scans the entire headline_category column and returns a vector containing only the distinct values, removing all duplicates. The length() function then counts how many elements are in that vector. The sort() function is optional and displays all category names in alphabetical order for easier reading.

Assumptions

- Each subcategory (e.g., 'news.law.courts.circuit-court') is treated as a separate unique category, not grouped under its parent category.
- No NA values exist in the headline_category column.

Question 2: Highest Mean Engagement & Lowest Article Count

Problem Statement

Which provider has the highest mean engagement? Which headline category has the lowest number of articles?

Approach

Part A: Group data by news_provider, compute the mean engagement_score for each, and find the maximum. Part B: Count articles per headline_category using table() and find the minimum count.

R Code

Part A: Provider with Highest Mean Engagement

```
# Calculate mean engagement score for each provider
provider_mean <- ds %>%
  filter(!is.na(engagement_score)) %>%
  group_by(news_provider) %>%
  summarise(mean_engagement = mean(engagement_score), .groups = "drop") %>%
  arrange(desc(mean_engagement))

# Display all providers sorted by mean engagement
print(provider_mean)

# Show the provider with highest mean engagement
provider_mean[1, ]
```

Part B: Category with Lowest Number of Articles

```
# Count articles per category
freq <- table(ds$headline_category)

# Get all categories with the lowest count (handles ties)
names(freq[freq == min(freq)])

# Show category name and its count
freq[freq == min(freq)]
```

Explanation

For Part A, we first remove rows where engagement_score is NA to avoid incorrect calculations. Then we group the data by news_provider and compute the mean. The arrange(desc()) function sorts from highest to lowest, so the first row is the answer.

For Part B, table() creates a frequency table counting how many articles exist per category. We use min(freq) to find the smallest count and then filter for all categories that match this minimum, which handles the case where multiple categories are tied for the lowest count.

Assumptions

- NA values in engagement_score are excluded when computing the mean (not treated as zero).
- Multiple categories may share the lowest article count (ties are reported).

Question 3: Article Counts by Category & Provider

Problem Statement

Compute the total number of articles for each headline category and news provider. Then, use a single R function to display Min, Max, and Mean of the total number of articles for each news provider.

Approach

Step 1 (data preparation): Use table() to create a cross-tabulation of news_provider and headline_category, then convert to a dataframe. Step 2 (single function): Use tapply() with summary to compute and display statistics grouped by provider.

R Code

```
# Step 1: Count total articles for each combination of provider and category
article_counts <- as.data.frame(table(ds$news_provider, ds$headline_category))
colnames(article_counts) <- c("news_provider", "headline_category",
"total_articles")

# Step 2: Display Min, Max, Mean using a SINGLE function
tapply(article_counts$total_articles, article_counts$news_provider, summary)
```

Explanation

The table() function creates a contingency table counting articles for every combination of news_provider and headline_category. We convert this to a dataframe with as.data.frame() and rename the columns for clarity.

The tapply() function is the key single function required by the question. It takes three arguments: the data vector (total_articles), the grouping factor (news_provider), and the function to apply (summary). The summary function returns Min, 1st Quartile, Median, Mean, 3rd Quartile, and Max — covering all requested statistics.

Assumptions

- The question allows multiple functions for data preparation (Step 1) but requires exactly one function for the statistical display (Step 2).
- tapply(summary) is considered a single function call that displays all required statistics.

Question 4: Maximum Engagement Year & Largest Increase

Problem Statement

In which year did TheJournal.ie record its maximum engagement score? Which news provider shows the largest increase in engagement score over the years?

Approach

Part A: Filter data for TheJournal.ie, find the row with maximum engagement_score, and extract its year. Part B: For each provider, compute yearly mean engagement scores, fit a linear regression (mean_score ~ year), and compare slopes. The provider with the highest positive slope has the largest increase.

R Code

```
library(dplyr)
library(lubridate)

# Extract year from publish_date
ds$year <- as.numeric(format(dmy(gsub("(\\d+)(st|nd|rd|th)", "\\1",
ds$publish_date)), "%Y"))

# Part A: Year of maximum engagement for TheJournal.ie
journal_data <- ds %>%
  filter(news_provider == "TheJournal.ie")

journal_data$year[which.max(journal_data$engagement_score)]

# Part B: Provider with largest increase over years
yearly_avg <- ds %>%
  group_by(news_provider, year) %>%
  summarise(mean_score = mean(engagement_score, na.rm = TRUE), .groups = "drop")

# Fit linear model for each provider, extract slope
slopes <- yearly_avg %>%
  group_by(news_provider) %>%
  summarise(slope = coef(lm(mean_score ~ year))[2], .groups = "drop") %>%
  arrange(desc(slope))

# First row = provider with largest increase
slopes[1, ]
```

Explanation

Date parsing: The gsub() function removes ordinal suffixes (st, nd, rd, th) from dates. Then dmy() from lubridate parses the cleaned string into a Date object, and format("%Y") extracts just the year.

Part A: which.max() returns the index of the row containing the maximum engagement_score. We use this index to look up the corresponding year.

Part B: We compute the yearly average engagement score per provider. Then for each provider, we fit a linear regression model (lm) where mean_score is predicted by year. The slope (second coefficient) represents the rate of change per year. A higher positive slope means a larger increase over time. We sort by slope descending to find the provider with the greatest increase.

Assumptions

- "Largest increase" is interpreted as the steepest positive linear trend in yearly mean engagement scores, measured by the regression slope.
- If TheJournal.ie has multiple rows tied for the maximum engagement score, which.max() returns the first occurrence.

Question 5: Factors Associated with Engagement Scores

Problem Statement

Investigate the factors associated with higher or lower engagement scores in the given dataset.

Approach

We analyse four potential factors: (1) Headline Category, (2) News Provider, (3) Time/Year, and (4) Headline Length. For each factor, we compute summary statistics, create visualisations, and run statistical tests including ANOVA and multiple regression.

R Code

```
library(dplyr)
library(lubridate)
library(ggplot2)

# Remove rows with NA engagement scores
ds_clean <- ds %>% filter(!is.na(engagement_score))

# ---- Factor 1: Headline Category ----
ds_clean %>%
  group_by(headline_category) %>%
  summarise(
    mean_score = mean(engagement_score),
    median_score = median(engagement_score),
    sd_score = sd(engagement_score),
    count = n()
  ) %>%
  arrange(desc(mean_score))

ggplot(ds_clean, aes(x = reorder(headline_category, engagement_score, FUN =
median),
                y = engagement_score)) +
  geom_boxplot() + coord_flip() +
  labs(title = "Engagement Score by Category", x = "Category", y = "Score")

# ---- Factor 2: News Provider ----
ds_clean %>%
  group_by(news_provider) %>%
  summarise(
    mean_score = mean(engagement_score),
    median_score = median(engagement_score),
    sd_score = sd(engagement_score),
    count = n()
  ) %>%
  arrange(desc(mean_score))

ggplot(ds_clean, aes(x = reorder(news_provider, engagement_score, FUN = median),
```

```
               y = engagement_score)) +
  geom_boxplot() + coord_flip() +
  labs(title = "Engagement Score by Provider", x = "Provider", y = "Score")

# ---- Factor 3: Time Trend ----
ds_clean$date_parsed <- dmy(gsub("(\\d+)(st|nd|rd|th)", "\\1",
ds_clean$publish_date))
ds_clean$year <- year(ds_clean$date_parsed)

yearly_trend <- ds_clean %>%
  group_by(year) %>%
  summarise(mean_score = mean(engagement_score))

ggplot(yearly_trend, aes(x = year, y = mean_score)) +
  geom_line() + geom_point() +
  labs(title = "Yearly Trend of Engagement Score", x = "Year", y = "Mean Score")

# ---- Factor 4: Headline Length ----
ds_clean$headline_length <- nchar(ds_clean$headline_text)
cor.test(ds_clean$headline_length, ds_clean$engagement_score)

ggplot(ds_clean, aes(x = headline_length, y = engagement_score)) +
  geom_point(alpha = 0.3) + geom_smooth(method = "lm") +
  labs(title = "Headline Length vs Engagement", x = "Length", y = "Score")

# ---- ANOVA Tests ----
summary(aov(engagement_score ~ headline_category, data = ds_clean))
summary(aov(engagement_score ~ news_provider, data = ds_clean))

# ---- Multiple Regression ----
model <- lm(engagement_score ~ headline_category + news_provider + year +
headline_length, data = ds_clean)
summary(model)
```

Explanation

Factor 1 (Category): We compute mean, median, and standard deviation for each headline category. The boxplot visualises the distribution. Categories with higher median scores attract more engagement.

Factor 2 (Provider): Same approach as Factor 1, but grouped by news provider. This reveals whether certain publishers consistently achieve higher engagement.

Factor 3 (Year): We track how the mean engagement score evolves over time. The line chart shows temporal trends, such as whether engagement is increasing or decreasing over the years.

Factor 4 (Headline Length): We compute the Pearson correlation between headline character count and engagement score. The scatter plot with regression line shows the direction and strength of any linear relationship.

ANOVA tests determine whether the differences between group means (categories, providers) are statistically significant (p-value < 0.05). The multiple regression model combines all factors to identify which ones are the strongest predictors of engagement, controlling for the others.

Assumptions

- Rows with NA engagement_score are removed before all analyses to avoid warnings and incorrect results.
- Headline length is measured in characters (nchar), not words.
- ANOVA assumes approximately normal distributions and homogeneity of variance within groups.

Question 6: Headline Categories, Engagement & Time

Part A: Top 3 Categories by Correlation

Problem Statement

For each news provider, identify the top 3 headline categories with the strongest association between yearly article volume and yearly mean engagement score, based on correlation analysis.

R Code

```
library(dplyr)

yearly_data <- ds %>%
  group_by(news_provider, headline_category, year) %>%
  summarise(
    yearly_count = n(),
    yearly_mean_score = mean(engagement_score, na.rm = TRUE),
    .groups = "drop"
  )

cor_results <- yearly_data %>%
  group_by(news_provider, headline_category) %>%
  filter(n() >= 3) %>%
  summarise(
    correlation = cor(yearly_count, yearly_mean_score, use = "complete.obs"),
    p_value = cor.test(yearly_count, yearly_mean_score)$p.value,
    .groups = "drop"
  )

top3_cor <- cor_results %>%
  group_by(news_provider) %>%
  arrange(desc(abs(correlation))) %>%
  slice_head(n = 3)

print(top3_cor)
```

Explanation

We first aggregate the data to get yearly article counts and yearly mean engagement scores for every provider-category combination. Then for each combination (with at least 3 years of data), we compute the Pearson correlation coefficient between yearly_count and yearly_mean_score. The abs() function ensures we rank by strength of association regardless of direction (positive or negative). The top 3 per provider are selected using slice_head().

Part B: Top 3 Categories with Increasing Trends

Problem Statement

For each news provider, identify the top 3 headline categories with the most significant increasing trends in yearly article counts.

R Code

```
yearly_counts <- ds %>%
  group_by(news_provider, headline_category, year) %>%
  summarise(yearly_count = n(), .groups = "drop")

trend_results <- yearly_counts %>%
  group_by(news_provider, headline_category) %>%
  filter(n() >= 3) %>%
  summarise(
    slope = coef(lm(yearly_count ~ year))[2],
    p_value = summary(lm(yearly_count ~ year))$coefficients[2, 4],
    .groups = "drop"
  )

top3_trend <- trend_results %>%
  filter(slope > 0) %>%
  group_by(news_provider) %>%
  arrange(p_value) %>%
  slice_head(n = 3)

print(top3_trend)
```

Explanation

For each provider-category pair, we fit a linear regression of yearly article count on year. The slope indicates direction: positive means the category is producing more articles over time. The p-value indicates statistical significance: a lower p-value means the increasing trend is more reliable. We filter for positive slopes only (increasing trends), then select the top 3 most statistically significant per provider by sorting on p-value ascending.

Assumptions (Both Parts)

- A minimum of 3 years of data is required to compute a meaningful correlation or trend (filter(n() >= 3)).
- Pearson correlation is used, which assumes a linear relationship.
- "Most significant" in Part B refers to the lowest p-value (most statistically significant), not necessarily the largest slope.

Question 7: Period-Based Analysis (2019-2020)

Problem Statement

Select data for 2019 and 2020, create a Period column based on quarterly date ranges (Period 1 through Period 8), compute total articles by period and category, and generate a boxplot.

R Code

```
library(dplyr)
library(lubridate)
library(ggplot2)

ds$date_parsed <- dmy(gsub("(\\d+)(st|nd|rd|th)", "\\1", ds$publish_date))

# Filter for 2019 and 2020
ds_filtered <- ds %>% filter(year(date_parsed) %in% c(2019, 2020))

# Create Period column based on quarterly date ranges
ds_filtered <- ds_filtered %>%
  mutate(Period = case_when(
    date_parsed >= as.Date("2019-01-01") & date_parsed <= as.Date("2019-03-31") ~
"Period 1",
    date_parsed >= as.Date("2019-04-01") & date_parsed <= as.Date("2019-06-30") ~
"Period 2",
    date_parsed >= as.Date("2019-07-01") & date_parsed <= as.Date("2019-09-30") ~
"Period 3",
    date_parsed >= as.Date("2019-10-01") & date_parsed <= as.Date("2019-12-31") ~
"Period 4",
    date_parsed >= as.Date("2020-01-01") & date_parsed <= as.Date("2020-03-31") ~
"Period 5",
    date_parsed >= as.Date("2020-04-01") & date_parsed <= as.Date("2020-06-30") ~
"Period 6",
    date_parsed >= as.Date("2020-07-01") & date_parsed <= as.Date("2020-09-30") ~
"Period 7",
    date_parsed >= as.Date("2020-10-01") & date_parsed <= as.Date("2020-12-31") ~
"Period 8"
  ))

# Total articles by Period and category
period_counts <- ds_filtered %>%
  group_by(Period, headline_category) %>%
  summarise(total_articles = n(), .groups = "drop")

# Boxplot
ggplot(period_counts, aes(x = Period, y = total_articles)) +
  geom_boxplot(fill = "lightblue") +
  labs(title = "Distribution of Total Articles by Period (2019-2020)",
       x = "Period", y = "Total Number of Articles") +
  theme_minimal()
```

Explanation

We first filter the dataset for only 2019 and 2020. The case_when() function acts like a series of if-else statements, assigning each row a Period label based on which quarterly date range it falls into. Periods 1-4 cover Q1-Q4 of 2019, and Periods 5-8 cover Q1-Q4 of 2020.

After labelling, we count articles per Period-Category combination. The boxplot shows how these counts are distributed across categories within each period, revealing seasonal patterns and the potential impact of COVID-19 (Periods 5-8).

Assumptions

- Each period covers exactly one calendar quarter (3 months).
- Dates falling outside 2019-2020 are excluded by the initial filter.

Question 8: RTE News September Trend

Problem Statement

Examine the trend in the number of articles published by RTE News in September over the years and create an appropriate chart.

R Code

```
library(dplyr)
library(lubridate)
library(ggplot2)

ds$date_parsed <- dmy(gsub("(\\d+)(st|nd|rd|th)", "\\1", ds$publish_date))
ds$year <- year(ds$date_parsed)
ds$month <- month(ds$date_parsed)

# Filter for RTE News in September
rte_september <- ds %>%
  filter(news_provider == "RTE News", month == 9)

# Count articles per year
rte_sept_yearly <- rte_september %>%
  group_by(year) %>%
  summarise(total_articles = n(), .groups = "drop")

# Line chart with trend line
ggplot(rte_sept_yearly, aes(x = year, y = total_articles)) +
  geom_line(color = "blue", linewidth = 1) +
  geom_point(color = "red", size = 2) +
  geom_smooth(method = "lm", se = TRUE, linetype = "dashed", color = "grey") +
  labs(title = "RTE News Articles Published in September Over the Years",
       x = "Year", y = "Total Articles") +
  scale_x_continuous(breaks = seq(min(rte_sept_yearly$year),
                                  max(rte_sept_yearly$year), by = 1)) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

Explanation

We filter the dataset for articles published by RTE News in month 9 (September). Then we count how many articles were published each year. The line chart shows the actual counts with blue lines and red dots. The grey dashed line is a linear regression trend line with a confidence interval (se = TRUE), showing the overall direction of the trend.

The scale_x_continuous() ensures every year is labelled on the x-axis, and angle = 45 rotates these labels to prevent overlapping.

Assumptions

- A line chart with a trend line is an appropriate choice for showing temporal trends.
- The news_provider name is exactly 'RTE News' in the dataset.

Question 9: External Factors Influencing Article Trends

Problem Statement

Using both the original dataset and external datasets, investigate the factors influencing the yearly trend in the number of articles published by some news providers. Analyse at least 20 years of data.

Approach

We select news providers with at least 20 years of data. We then investigate multiple factors: major Irish events, category composition changes, engagement score relationship, year-over-year growth rates, internet penetration, and GDP. External data is sourced from the World Bank.

Key Analyses Performed

6. Yearly article count trends per provider with linear regression.
7. Major Irish events overlay (9/11, Financial Crisis, IMF Bailout, Brexit, COVID-19, etc.) to identify event-driven spikes.
8. Content category composition analysis using stacked area charts to show editorial shifts.
9. Engagement score vs. article volume correlation per provider.
10. Year-over-year percentage growth rate analysis.
11. External factor: Irish internet penetration (% of population) from World Bank.
12. External factor: Irish GDP (Billion USD) from World Bank.

External Data Sources

Source	Indicator	URL
World Bank	Internet Users (% pop.)	data.worldbank.org/indicator/IT.NET.USER.ZS
World Bank	GDP (current USD)	data.worldbank.org/indicator/NY.GDP.MKTP.CD
Wikipedia	Major Irish Events	en.wikipedia.org/wiki/2000s_in_Ireland

Key R Code Highlights

Selecting Providers with 20+ Years

```
provider_years <- ds %>%
  group_by(news_provider) %>%
  summarise(
    min_year = min(year, na.rm = TRUE),
    max_year = max(year, na.rm = TRUE),
    year_span = max_year - min_year + 1,
    .groups = "drop"
  ) %>%
  filter(year_span >= 20)
```

Main Category Extraction

```
# Group 80+ subcategories into main categories (business, news, sport, etc.)
ds_selected$main_category <- sapply(
  strsplit(ds_selected$headline_category, "\\."), function(x) x[1]
)
```

Merging External Data

```
# Use inner_join to merge and automatically exclude unmatched years
merged_internet <- yearly_articles %>%
  inner_join(internet_data, by = "year")
```

Correlation Summary

```
# Compute correlation between external factor and article volume per provider
internet_cor <- merged_internet %>%
  group_by(news_provider) %>%
  summarise(
    correlation = cor(total_articles, internet_pct, use = "complete.obs"),
    p_value = cor.test(total_articles, internet_pct)$p.value,
    .groups = "drop"
  )
```

Explanation

Provider Selection: We calculate the year span for each provider and filter for those with at least 20 years of data, as required by the question.

Category Simplification: Since there are 80+ subcategories, we extract just the first part of the hierarchical name (before the first dot) to create main categories like 'business', 'news', 'sport', etc. This makes the stacked area chart readable.

Event Overlay: We manually create a dataframe of major Irish/global events with their years. The geom_vline() function draws vertical dashed lines at each event year, and geom_text() labels them. This allows visual inspection of whether article volumes spike around these events.

External Data: Internet penetration and GDP data are sourced from World Bank. We use inner_join() (instead of left_join()) to merge this data with yearly article counts, which automatically drops unmatched rows and prevents the 'Removed rows' warning.

Growth Rate: Year-over-year percentage change is calculated using lag() to compare each year with the previous year. Values are filtered for NA and Inf before plotting.

Assumptions

- External data values used in the code are approximate. For production use, actual CSV files should be downloaded from World Bank and loaded with read.csv().

- Linear relationships are assumed when computing correlations between external factors and article volume.
- Correlation does not imply causation; external factors may be associated with but not directly causing changes in article volume.
- inner_join() is used instead of left_join() to avoid NA values and plotting warnings from unmatched years.